

Summary of module 05

In Module 05 we covered the following topics:

- Dictionary Collections
 - Adding and removing files
 - Using dictionaries with file data
- Json files
 - reading and writing
- Structured Error Handling
- Git hub
-

Steps to complete assignment 05

The Assignment:

Create a Python program that demonstrates using constants, variables, and print statements to display a message about a student's registration for a Python course. This program is very similar to Assignment04, but It adds the use of data processing using dictionaries and exception handling.

1. Build the program file from the information we did know based on the assignment criteria starting with the file Assignment05-Starter.py.
2. Update the header information
3. Import the .json file
4. Define constants
 - Our output file is to be Enrolments.json
 - Menu constant provided in script
 - Added constants to change text color of warnings
5. Define variables
6. Open a file to read into memory as a list (.json)
 - Remove the old code pointing to the .csv and
 - Added Error handling code to let the end user know if they are missing the file

```

42 # Extract the data from the file with error handling
43 try:
44     file = open(FILE_NAME, "r") # opens the .json file
45     students = json.load(file) # set the imported data to a variable "students"
46 except FileNotFoundError as e: # creates an error if there is not .json file
47     print("Text file must exist before running this script!\n")
48     print(RED + "-- Technical Error Message -- " + RESET)
49     print(e, e.__doc__, type(e), sep='\n')
50     print("program ended") # since there is no file end program
51     sys.exit(1)
52 except Exception as e:
53     print("There was a non-specific error!\n")
54     print(RED + "-- Technical Error Message -- " + RESET)
55     print(e, e.__doc__, type(e), sep='\n')
56
57 finally: # Check if a file object exists and is still open
58     if file is not None and file.closed == False:
59         file.close()

```

Figure 1 Importing the .json with error handling

7. Create a While loop, print the menu and start filling out the code for our menu choices 1-4 adding error handling for input errors
 - Choice 1 – take user input for registering for a course

```

69 # Input user data
70 if menu_choice == "1":
71     try:
72         student_first_name = input("Enter the student's first name: ")
73         if not student_first_name.isalpha(): # error if user enters a number
74             raise ValueError(RED + "The first name should not contain numbers." + RESET)
75         student_last_name = input("Enter the student's last name: ")
76         if not student_last_name.isalpha(): # error if user enters a number
77             raise ValueError(RED + "The first name should not contain numbers." + RESET)
78         course_name = input("Please enter the name of the course: ")

```

Figure 2 example of the error handling in the While loop

- Choice 2 – Show current data – what was read in from file and any inputted
 - Adapt old code to format correctly for the dictionary .json file reading from data in memory. We just have to print each row instead of formatting the indices.

```

71 for student in students:
72     print(f"Student {student[0]} {student[1]} is enrolled in {student[2]}")

102 for each_row in students:
103     #print(each_row)
104     print(each_row["FirstName"], each_row["LastName"], "is registered for", each_row["CourseName"])

```

Figure 3 A and B shows the indices(A) vs the catalog row print

- Choice 3 – write (dump) the new combined data to a .json document
 - Added error handling if there is no .json file to write to (fig 4 line 121)

```
107     # Save the data to a file
108     elif menu_choice == "3":
109         try:
110             file = open(FILE_NAME, "w")
111             json.dump(students, file)
112             print("----Data Saved----")
113         except TypeError as e:
114             print("Please check that the data is a valid JSON format\n")
115             | print("-- Technical Error Message -- ")
116             | print(e, e.__doc__, type(e), sep='\n')
117         except Exception as e:
118             print("-- Technical Error Message -- ")
119             print("Built-In Python error info: ")
120             print(e, e.__doc__, type(e), sep='\n')
121         finally:
122             if file is not None and file.closed == False:
123                 file.close()
124         continue
```

Figure 4 – New code to

- Choice 4 – end the program

```
127     elif menu_choice == "4":
128         | break # out of the loop
129     else:
130         | print(RED + "Please only choose option 1, 2, 3, or 4"+ RESET)
131
132     print("Program Ended")
```

8. Testing

- Noy only was testing needed for the program running as expected but also for the program to run the error handling. Here are some examples of the error handling working when incorrect data in entered.

```
What would you like to do? a
Please only choose option 1, 2, 3, or 4
```

Figure 5 – error handling when a letter is inputted instead of a number

```
What would you like to do? 1
Enter the student's first name: john3
The first name should not contain numbers.
-- Technical Error Message --
Inappropriate argument value (of correct type).
The first name should not contain numbers.
```

Figure 6 – error handling when a number is inputted instead of letters

9. Final the assignment was uploaded to Git

Summary

In Module 05 assignment we learned how to work with .json files and error handling and the Git repository. I found this assignment went smoother for me then the past. Perhaps I'm getting the flow of things, but defiantly am able to see the larger picture at this stage and not caught up so much in worrying about internalizing every character. I'm more comfortable with seeking out answers outside of the class. I have used Git at my previous job so I felt comfortable with that portion of this assignment.